

Case Study 12 — Frequency-Resolved Prefactor

Switching the oscillator-strength $\rightarrow \varepsilon$ convention

Contents

1	Case Study 12 — Frequency-Resolved Prefactor	1
1.1	What you will produce	1
1.2	1. Prerequisites	2
1.3	2. Inputs	2
1.4	3. Run with the CLI	2
1.5	4. Run with the HTTP API	2
1.6	5. Drive both interfaces from one Python script	2
1.7	6. Expected output layout	2
1.8	7. What to compare	3
1.9	8. Related case studies	3

1 Case Study 12 — Frequency-Resolved Prefactor

Switch the oscillator-strength $\rightarrow \varepsilon$ convention from the default **constant** mode to **frequency-resolved**. In the frequency-resolved mode the wavenumber factor is kept inside the integrand:

$$\varepsilon(\nu) = \sum_i P f_i \frac{\nu}{\nu_i} g_i(\nu - \nu_i).$$

The two modes produce identical integrated band areas for symmetric lineshapes but differ in peak height and peak position by $O((\sigma/\nu_i)^2)$ — below $\sim 2\%$ over 200–800 nm and growing into the NIR.

The user-selectable knob is:

- CLI: `--prefactor-mode frequency-resolved`
- API: form field `prefactor_mode=frequency-resolved`

1.1 What you will produce

Everything is written under one `output/` directory inside this case study folder:

Folder	By	Contents
<code>output/cli/tables/</code>	CLI	Result tables (CSV / JSON / XLSX)
<code>output/cli/plots/</code>	CLI	TIFF plots per sample, light source, and (sample, light) pair
<code>output/api/</code>	API	Same <code>tables/</code> and <code>plots/</code> tree, extracted from the ZIP
<code>output/api/analysis_outputs.zip</code>	API	Raw ZIP response

Light sources used: **AM1.5G** + **LED-B4**.

1.2 1. Prerequisites

If you have never run any case study before, follow [Case Study 01 §1 Prerequisites](#) once to install the package, the `requests` library, and either the Python or Docker option for the HTTP API. Every case study uses the same environment.

1.3 2. Inputs

The case-study `input.json` lists 2 Gaussian TD-DFT outputs. The manifest uses relative paths that point at the bundled data under `../input/files/`, so no files are copied or duplicated. To swap in your own data, replace `input.json` with a manifest of the same shape; you do not need to touch `submit.py`.

1.4 3. Run with the CLI

Open a terminal at the repository root, then run a single command:

```
overlap-calculator analyze --input case_studies/12_frequency_resolved_prefactor/input.json --out
↪ case_studies/12_frequency_resolved_prefactor/output/cli --default-light-sources AM15G,LEDB4
↪ --prefactor-mode frequency-resolved --plot-dpi 400
```

After the run, `case_studies/12_frequency_resolved_prefactor/output/cli/` contains a `tables/` directory and a `plots/` directory. The `results.*` tables carry a `prefactor_mode` column confirming the value used. To compare directly with the default, run the same command without `--prefactor-mode frequency-resolved` and diff the two `descriptor_summary.*` files.

1.5 4. Run with the HTTP API

Start the API as documented in [Case Study 01 §1](#) and verify with `curl http://localhost:8000/health` (or `:8080` if you started the API via `docker compose`). Then submit the same uploads in one `curl` command:

```
curl -X POST http://localhost:8000/analyze -F "files=@input/files/slurm-2829207.out" -F
↪ "files=@input/files/slurm-2829212.out" -F "default_light_sources=AM15G,LEDB4" -F
↪ "prefactor_mode=frequency-resolved" -F "plot_dpi=400" --output
↪ case_studies/12_frequency_resolved_prefactor/output/api/analysis_outputs.zip
```

Each `-F "files=@..."` is one upload. The form fields mirror the CLI flags. The response is a ZIP archive saved as `output/api/analysis_outputs.zip`.

1.6 5. Drive both interfaces from one Python script

Instead of typing the commands above, run `submit.py` from the repository root:

```
python case_studies/12_frequency_resolved_prefactor/submit.py
```

The script reads `input.json`, runs the CLI step (writing to `output/cli/`), then POSTs the same uploads to `/analyze` and unpacks the response into `output/api/`. If the API is not reachable, it prints a hint and exits cleanly without failing the CLI step.

1.7 6. Expected output layout

```
case_studies/12_frequency_resolved_prefactor/output/
+-- cli/
|   +-- run_manifest.json
|   +-- tables/results.{csv,json,xlsx}          (2 samples x 2 light sources = 4 rows)
|   +-- tables/results_timings.{csv,json,xlsx}
```

```
| +-- tables/descriptor_summary.{csv,xlsx}  
| +-- tables/ranking_by_light_source__<metric>.{csv,json,xlsx}  
| +-- tables/ranking_by_sample__<metric>.{csv,json,xlsx}  
| +-- plots/  
+-- api/  
    +-- analysis_outputs.zip  
    +-- tables/...  
    +-- plots/...
```

Open `tables/results.csv` and inspect the `prefactor_mode` column: every row should show `frequency-resolved`. The `run_manifest.json` at the output root records `"prefactor_mode": "frequency-resolved"` in its `parameters` block.

1.8 7. What to compare

- Peak molar absorptivity (`gaussian_molar_absorptivity_max`) will be slightly higher than the constant-prefactor run for the same `sigma_ev` because the wavenumber factor shifts weight toward higher frequencies.
 - For visible dyes the difference is typically below 2%; for NIR bands it grows.
-

1.9 8. Related case studies

- [Case Study 08 — Broadening Sigma](#)
- [Case Study 13 — Marcus-Hush Width](#)